

PyQCU 中 CUDA C++ 版 MultiGrid 求解器的设计与优化解析

张 Xin

2026 年 7 月 17 日

目录

1	引言	3
2	数学基础: Clover-improved Wilson Dirac 算子	3
2.1	Wilson Dirac 算子	3
2.2	Clover 改进项	4
2.3	数据布局	4
3	Even-Odd 预条件	4
3.1	数学原理	4
3.2	CUDA 实现	5
4	BiStabCG 求解器的 CUDA 多流优化	5
4.1	BiCGStab 算法	5
4.2	CUDA 多流并行优化	6
4.3	Wilson Dslash 的 CUDA Kernel 设计	7
4.4	MPI 通信与计算的 Overlap	8
5	MultiGrid 层级构造	8
5.1	整体架构	8
5.2	层级参数	9
5.3	近零空间生成 (give_null_vecs)	9
5.4	局域正交化 (local_orthogonalize)	10
5.5	Galerkin 投影构建粗网格算子	10
5.6	延拓与限制操作	11
6	V-cycle 递归求解与自适应策略	11
6.1	V-cycle 递归结构	11
6.2	自适应 Level-Back 机制	12

7	CUDA C++ MultiGrid 参数体系	12
7.1	MultiGrid 相关参数索引	12
8	Python-CUDA 桥接层 (Cython)	13
8.1	Cython 接口设计	13
8.2	参数传递	14
8.3	Plan 系统	14
8.4	生命周期管理	14
8.5	MultiGrid 中的 CUDA 调用	15
9	性能优化策略总结	15
9.1	CUDA Kernel 级优化	15
9.2	MPI 通信优化	15
9.3	算法级优化	16
9.4	内存管理优化	16
10	代码结构导航	16
11	总结与展望	16

1 引言

格点量子色动力学 (Lattice QCD) 中, 求解大规模稀疏线性系统 $D\psi = \chi$ 是计算最密集的环节之一。Dirac 算子 D 的条件数随夸克质量减小而急剧增长, 导致传统 Krylov 子空间方法 (如 CG、BiCGStab) 收敛缓慢。代数多重网格 (Algebraic Multigrid, AMG) 方法通过构造近零空间 (near-null space) 和层级粗化算子, 能够显著克服临界减速 (critical slowing down) 问题, 在物理夸克质量下保持接近常数的收敛速率。

PyQCU 是一个基于 Python/Cython 与 CUDA 的格点 QCD 计算框架, 其 MultiGrid 求解器采用 “Python 驱动 + CUDA C++ 加速” 的混合架构:

- **Python 层:** 负责层级构造、近零空间生成、Galerkin 粗化、延拓/限制操作以及 V-cycle 控制流;
- **CUDA C++ 层:** 负责最细层级上 Wilson/Clover Dirac 算子的高性能计算、BiCGStab 迭代求解以及 MPI 多 GPU 通信。

本文对该 MultiGrid 求解器的完整设计与优化策略进行系统性解析, 内容涵盖:

1. Clover-improved Wilson Dirac 算子的数学形式与 CUDA 实现;
2. Even-Odd 预条件 Schur 补系统的构造;
3. BiStabCG (BiCGStab) 求解器的 CUDA 多流并行优化;
4. 近零空间向量的生成与局域正交化;
5. Galerkin 投影构建粗网格算子;
6. 多层次 V-cycle 递归求解与自适应 Level-back 机制;
7. MPI 多 GPU 通信的 Overlap 优化;
8. Cython 桥接层的设计。

2 数学基础: Clover-improved Wilson Dirac 算子

2.1 Wilson Dirac 算子

连续 QCD 中的 Dirac 算子为 $D = \gamma_\mu D_\mu + m$ 。在格点上, 使用 Wilson 离散化方案:

$$D_W(x, y) = \delta_{x,y} - \kappa \sum_{\mu=0}^3 \left[(1 - \gamma_\mu) U_{x,\mu} \delta_{x+\hat{\mu},y} + (1 + \gamma_\mu) U_{x-\hat{\mu},\mu}^\dagger \delta_{x-\hat{\mu},y} \right] \quad (1)$$

其中 $\kappa = 1/(2m + 8)$ 为 hopping 参数, $U_{x,\mu} \in SU(3)$ 为规范场链路变量, γ_μ 为 4×4 Dirac gamma 矩阵。在 PyQCU 代码中, 该算子实现于 `pyqcu/dslash/_wilson.py` 的 `give_wilson` 函数。

2.2 Clover 改进项

为消除 $\mathcal{O}(a)$ 的格点离散化误差，加入 Sheikholeslami-Wohlert (Clover) 项：

$$D_C = D_W - \frac{a^2 \kappa}{u_0^4} \sum_{\mu < \nu} \sigma_{\mu\nu} F_{\mu\nu} \delta_{x,y} \quad (2)$$

其中 $\sigma_{\mu\nu} = -\frac{i}{2}[\gamma_\mu, \gamma_\nu]$ ，场强张量 $F_{\mu\nu}$ 由四个 link 的 clover plaquette 构造：

$$F_{\mu\nu} = \frac{1}{8i} \left[U_{x,\mu} U_{x+\hat{\mu},\nu} U_{x+\hat{\nu},\mu}^\dagger U_{x,\nu}^\dagger \right. \quad (3)$$

$$+ U_{x,\nu} U_{x-\hat{\mu}+\hat{\nu},\mu}^\dagger U_{x-\hat{\mu},\nu}^\dagger U_{x-\hat{\mu},\mu} \quad (4)$$

$$+ U_{x-\hat{\mu},\mu}^\dagger U_{x-\hat{\mu}-\hat{\nu},\nu}^\dagger U_{x-\hat{\mu}-\hat{\nu},\mu} U_{x-\hat{\nu},\nu} \quad (5)$$

$$\left. + U_{x-\hat{\nu},\nu}^\dagger U_{x-\hat{\nu},\mu} U_{x-\hat{\nu}+\hat{\mu},\nu} U_{x,\mu}^\dagger - \text{h.c.} \right] \quad (6)$$

Clover 项在 PyQCU 中的 Python 实现见 `pyqcu/dslash/_clover.py::make_clover`, CUDA C++ 实现见 `cpp/cuda/qcu/include/lattice_clover_dslash.h::make_clover`。

最终完整的 Clover Dirac 算子写为 $A = I + T$ ，其中 T 为 Clover 项 (spin×color 空间中的 12×12 矩阵)， I 为单位矩阵。在代码中通过 `add_I` 和 `cut_I` 在两种形式之间转换。

2.3 数据布局

PyQCU 中张量的维度约定如下 (见表1)：

表 1: 关键张量的维度布局

张量	形状	索引含义
Gauge 场 U	[3,3,4,Lx,Ly,Lz,Lt]	[color,color,direction,x,y,z,t]
Fermion 场 ψ	[4,3,Lx,Ly,Lz,Lt]	[spin,color,x,y,z,t]
Clover 项 A	[4,3,4,3,Lx,Ly,Lz,Lt]	[spin,color,spin,color,x,y,z,t]
12 分量压缩形式	[12,12,Lx,Ly,Lz,Lt]	[spin×color, spin×color, x,y,z,t]
Even-Odd 分离	[2, ...]	[parity=even/odd, ...]

在 CUDA C++ 层中，数据进行了一维扁平化存储。每个格点位置的 12 个 spin-color 分量 (`_LAT_SC=12`) 连续存储，然后按格点索引排列。规范场每个格点存储 4 个方向的 SU(3) 矩阵，在 Dslash 计算中通过宏 `get_u` 加载当前方向上的 3×3 矩阵及反余子式重构的第三行元素。

3 Even-Odd 预条件

3.1 数学原理

将格点按照 $(x + y + z + t) \bmod 2$ 分为偶 (even) 和奇 (odd) 两个子集，Dirac 方程 $A\psi = \chi$ 可写为分块形式：

$$\begin{pmatrix} A_{ee} & A_{eo} \\ A_{oe} & A_{oo} \end{pmatrix} \begin{pmatrix} \psi_e \\ \psi_o \end{pmatrix} = \begin{pmatrix} \chi_e \\ \chi_o \end{pmatrix} \quad (7)$$

其中 A_{ee} 和 A_{oo} 仅包含对角 (Clover+ 质量) 项, A_{eo} 和 A_{oe} 包含 hopping 项。消去 ψ_e 得到关于 ψ_o 的 Schur 补系统:

$$\underbrace{(A_{oo} - A_{oe}A_{ee}^{-1}A_{eo})}_{\equiv \hat{A}} \psi_o = \underbrace{\chi_o - A_{oe}A_{ee}^{-1}\chi_e}_{\equiv \hat{b}} \quad (8)$$

该系统的条件数约为原系统的平方根, 显著改善了迭代求解的收敛性。

3.2 CUDA 实现

在 CUDA 层面, Even-Odd 预条件的 Dslash 操作由 dslash 成员函数实现 (lattice_clover_bistabcg.h:1

```
1 void dslash(void *fermion_out, void *fermion_in) {
2     // Step 1: D_eo * src_o (Wilson hopping, even<-odd)
3     wilson_dslash.run_eo(set_ptr->device_vec0, fermion_in, gauge);
4     // Step 2: A_ee^{-1} * (D_eo * src_o) (Clover inverse on even sites)
5     give_copy_vals<T><<<...>>>(set_ptr->device_vec2, set_ptr->device_vec0);
6     clover_dslash_ee_inv.give(set_ptr->device_vec2);
7     // Step 3: -kappa^2 * D_oe * (A_ee^{-1} * D_eo * src_o) (Wilson, odd<-even)
8     wilson_dslash.run_oe(set_ptr->device_vec1, set_ptr->device_vec2, gauge);
9     ;
10    // Step 4: A_oo * src_o + (-kappa^2 * D_oe * A_ee^{-1} * D_eo * src_o)
11    give_copy_vals<T><<<...>>>(set_ptr->device_vec2, fermion_in);
12    clover_dslash_oo.give(set_ptr->device_vec2);
13    bistabcg_give_dest_o<T><<<...>>>(
14        fermion_out, set_ptr->device_vec2, set_ptr->device_vec1, ...);
15 }
```

注意 CUDA 实现中使用了 Clover 逆矩阵 `clover_ee_inv` 作为输入 (由 Python 层计算并提供), 避免了每次迭代时重复求逆。 $\hat{A}$ 的操作涉及四次算子应用:

$$\hat{A} \cdot src_o = A_{oo} \cdot src_o - \kappa^2 D_{oe}(A_{ee}^{-1}(D_{eo}(src_o))) \quad (9)$$

其中 κ^2 因子出现在两次 hopping 操作各含一个 $-\kappa$ 因子。

4 BiStabCG 求解器的 CUDA 多流优化

4.1 BiCGStab 算法

BiStabCG (即 BiCGStab, 双共轭梯度稳定化方法) 用于在每个层级 (特别是最粗层级) 上求解线性系统。算法流程如算法1所示。

Algorithm 1 BiCGStab 求解器

Require: A, b , 初始猜测 x_0 , 容差 ε , 最大迭代次数 $N_{\max}$

Ensure: 近似解 x

```
1:  $r_0 \leftarrow b - Ax_0$ 
2: 选取  $\tilde{r}_0$  满足  $(\tilde{r}_0, r_0) \neq 0$  (代码中取  $\tilde{r}_0 = r_0$ )
3:  $\rho_0 \leftarrow \alpha \leftarrow \omega_0 \leftarrow 1$ 
4:  $v_0 \leftarrow p_0 \leftarrow 0$ 
5: for  $i = 1, 2, \dots, N_{\max}$  do
6:    $\rho_i \leftarrow (\tilde{r}_0, r_{i-1})$ 
7:    $\beta \leftarrow (\rho_i / \rho_{i-1})(\alpha / \omega_{i-1})$ 
8:    $p_i \leftarrow r_{i-1} + \beta(p_{i-1} - \omega_{i-1}v_{i-1})$ 
9:    $v_i \leftarrow Ap_i$  ▷ 矩阵向量乘
10:   $\alpha \leftarrow \rho_i / (\tilde{r}_0, v_i)$ 
11:   $s \leftarrow r_{i-1} - \alpha v_i$ 
12:   $t \leftarrow As$  ▷ 矩阵向量乘
13:   $\omega_i \leftarrow (t, s) / (t, t)$ 
14:   $x_i \leftarrow x_{i-1} + \alpha p_i + \omega_i s$ 
15:   $r_i \leftarrow s - \omega_i t$ 
16:  if  $\|r_i\| < \varepsilon$  then return  $x_i$ 
17:  end if
18: end for
```

4.2 CUDA 多流并行优化

PyQCU 的 CUDA C++ BiStabCG 实现在 `lattice_clover_bistabcg.h` 中, 采用四路 CUDA Stream 并行策略最大化 GPU 利用率。核心设计思想是将内积计算 (需要 MPI Allreduce 同步) 与向量更新 (纯本地操作) 分配到不同 stream 上并行执行。

每个四路 stream (`_a_`, `_b_`, `_c_`, `_d_`) 被分配不同的任务流:

- **Stream A** (`_a_`): 计算 $\rho = (\tilde{r}, r)$, 更新 $p = r + \beta(p - \omega v)$, 更新 $s = r - \alpha v$, 更新 $r = s - \omega t$;
- **Stream B** (`_b_`): 更新 $x = x + \alpha p + \omega s$;
- **Stream C** (`_c_`): 计算 $\|r\|^2$, 计算 (t, s) ;
- **Stream D** (`_d_`): 计算 $(\tilde{r}, v)$, 计算 (t, t) , 计算 $\omega = (t, s) / (t, t)$ 。

内积的 MPI Allreduce 采用 “cuBLAS Dot $\rightarrow$ D2H copy $\rightarrow$ MPI Barrier $\rightarrow$ MPI_Allreduce $\rightarrow$ H2D copy” 的流水线, 在 `_dot_mpi` 函数中实现。

此外, Dslash 计算 (包含 MPI halo 交换和内部格点计算) 在主 stream 上执行, 与其他四个计算 stream 通过 `cudaStreamSynchronize` 进行同步。

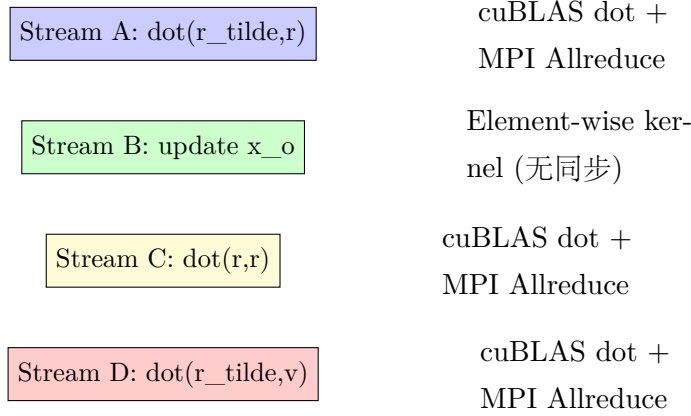


图 1: BiStabCG 迭代中的四路 CUDA Stream 流水线

4.3 Wilson Dslash 的 CUDA Kernel 设计

Wilson Dslash 的 CUDA kernel (`wilson_dslash.cu`) 每个线程处理一个格点, 在一个 kernel 内完成所有 4 个方向的 hopping 操作。核心循环结构为:

```

1  template <typename T>
2  __global__ void wilson_dslash(void *device_U, void *device_src,
3                               void *device_dest, void *device_params) {
4      int idx = blockIdx.x * blockDim.x + threadIdx.x;
5      // 从一维线程索引解码出 (x,y,z,t) 坐标
6      int x = idx / (lat_y * lat_z * lat_t);
7      int y = (idx % (lat_y * lat_z * lat_t)) / (lat_z * lat_t);
8      int z = (idx % (lat_z * lat_t)) / lat_t;
9      int t = idx % lat_t;
10
11     // 加载本格点的 src (12个 spin-color 分量)
12     get_src(src, origin_src, lat_xyz);
13
14     // 对每个方向: 加载 U 矩阵 → 加载邻居 src → SU(3) × spinor 乘法
15     // X-方向: (1+ _x) * U†(x-) * src(x-)
16     move_backward(move, x, lat_x);
17     tmp_U = origin_U + move*lat_y*lat_z*lat_t + offset_X;
18     get_u(U, tmp_U, lat_xyz);
19     // 计算 SU(3) 矩阵与 spinor 的乘积, 施加 (1± _ ) 投影
20     for (int c0 = 0; c0 < 3; c0++) {
21         for (int c1 = 0; c1 < 3; c1++) {
22             dest_spinor[c0] += U_dagger[c1*3+c0] * src_spinor[c1];
23         }
24     }
25     // ... 类似处理 X+, Y-, Y+, Z-, Z+, T-, T+
26 }
```

关键优化点:

- **SU(3) 矩阵压缩**: 每个 SU(3) 矩阵只存储前两行的 6 个复数, 第三行通过 $u_{ij}^* = \varepsilon_{ikl}\varepsilon_{jmn}u_{km}u_{ln}/2$ 从反余子式重构 (宏 `get_u` 中的后三行赋值), 节省了 1/3 的全局内存带宽;
- **寄存器内计算**: 每个线程将本格点的 12 个 spin-color 分量、邻居 src 分量和 SU(3) 矩阵元素全部加载到寄存器中, dest 也在寄存器中累加后统一写回;
- **Even-Odd 掩码**: 在 t 方向上的 Even-Odd 跳跃由宏 `move_forward_t` 和 `move_backward_t` 处理, 避免了分支发散。

4.4 MPI 通信与计算的 Overlap

Wilson Dslash 的 MPI 实现 (`lattice_wilson_dslash.h::run_mpi`) 采用以下流水线策略:

1. **阶段 1 —边界发送**: 四个方向 stream (`stream_dims[X/Y/Z/T]`) 同时启动边界数据 pack kernel, 将需要发送的 halo 数据打包到 send buffer; 完成后异步 D2H 拷贝;
2. **阶段 2 —内部计算**: 主 stream 启动内部格点 (不含边界) 的 Dslash kernel, 与阶段 1 的 D2H 拷贝并行;
3. **阶段 3 —MPI 通信**: 每个方向依次进行 MPI_Sendrecv (阻塞), 交换 halo 数据;
4. **阶段 4 —边界接收计算**: H2D 拷贝接收数据到 GPU, 启动边界接收 kernel 补全边界贡献。

该设计的性能优势在于: 内部格点计算 (占总计算的绝大部分) 与 halo 数据打包/D2H 拷贝重叠执行。也可切换到非阻塞 MPI 模式 (`run_mpi_non_block`) 使用 MPI_Isend/MPI_Irecv 配合 MPI_Wait, 进一步增大 overlap 窗口。

注意 Clover 项构建 (`lattice_clover_dslash.h::_make_mpi`) 涉及更复杂的通信模式: 除了一维 halo 交换 (每个方向单独通信), 还需要二维对角方向的 halo 交换 (如 xy、xz、xt、yz、yt、zt), 每个二维面需要 4 个通信 (BB、FB、BF、FF), 以支持跨角格点的 clover plaquette 计算。

5 MultiGrid 层级构造

5.1 整体架构

PyQCU 的 MultiGrid 求解器 (`pyqcu/solver/_multigrid.py`) 采用 V-cycle 递归结构。系统架构分为三个核心层次:

1. **Python 控制层** (`pyqcu/solver/_multigrid.py::multigrid`): 管理层级构造、参数配置、V-cycle 调度、自适应 level-back 策略和收敛监控;
2. **Python 算子层** (`pyqcu/dslash/_operator.py::operator`): 通过 Galerkin 投影构建粗网格上的 hopping 和 sitting (Clover) 算子;
3. **CUDA C++ 加速层**: 在最细层级提供高性能的 Dslash + BiCGStab 求解 (通过 Cython 桥接调用 `qcu.applyCloverBistabCgQcu`)。

5.2 层级参数

MultiGrid 的层级结构由以下参数控制：

- `min_size`: 每个方向的最小格点数，决定粗化的终止条件；
- `max_level`: 最大层级数（含最细层级）；
- `mg_grid_size = [bx, by, bz, bt]`: 每个粗网格块的大小，即粗化因子；
- `dof_list`: 每层的自由度（近零空间向量数目），典型值 `[12, 24, 24, 24, ...]`；
- `dtype_list`: 每层的数据类型（例如 `[complex64, complex128, complex128, ...]`）；
- `device_list`: 每层的计算设备（例如 `[cuda, cpu, cpu, ...]`）。

层级格点大小列表通过迭代粗化生成（`pyqcu/solver/_multigrid.py:33-36`）：

```
1 _lat_size = self.lat_size
2 while all(_ >= self.min_size for _ in _lat_size) and len(self.lat_size_list
3     ) < self.max_level:
4     self.lat_size_list.append(_lat_size)
5     _lat_size = [_lat_size[d] // self.mg_grid_size[d] for d in range(4)]
```

5.3 近零空间生成 (`give_null_vecs`)

近零空间向量（null-space vectors）是 AMG 方法的核心，它们张成 Dirac 算子低模本征空间的近似。生成过程（`pyqcu/tools/_multigrid.py::give_null_vecs`）为每个向量执行带逆的幂迭代：

$$v^{(i)} = v^{(i)} - A^{-1}Av^{(i)} \quad (10)$$

代码实现中，对每个随机初始化的向量 $v^{(i)}$ ：

1. 应用 $A \cdot v^{(i)}$ 得到右端向量；
2. 调用 BiCGStab 求解 $Ax = Av^{(i)}$ ，初始猜测为 $x_0 = v^{(i)}$ ；
3. $v^{(i)} \leftarrow v^{(i)} - x$ ，即 $v^{(i)} - A^{-1}Av^{(i)}$ ，这近似于投影到低模空间；
4. 可选地，对 $v^{(i)}$ 关于前面的向量做 Gram-Schmidt 正交化；
5. 归一化： $v^{(i)} \leftarrow v^{(i)} / \|v^{(i)}\|$ 。

如果提供了 CUDA 加速的 BiCGStab（`with_cuda_qcu=True`），最细层级使用 CUDA C++ 求解器；否则回退到纯 Python 的 BiCGStab。

5.4 局域正交化 (local_orthogonalize)

近零空间向量需要在每个粗网格块内局域正交化，以保证粗网格算子的稀疏性和良条件性。给定近零空间向量 $\phi^{(i)} \in \mathbb{C}^{[E, e, X \cdot x, Y \cdot y, Z \cdot z, T \cdot t]}$ ，其中 E 是粗自由度数目， e 是细自由度， (X, Y, Z, T) 是粗网格维度， (x, y, z, t) 是每个粗块内的细网格维度：

1. 将向量 reshape 为 $[X, Y, Z, T, E, e, x, y, z, t]$ ；
2. 置换维度和合并为 $[n_{\text{blocks}}, E, \text{local_dim}]$ ，其中 $n_{\text{blocks}} = X \times Y \times Z \times T$ ， $\text{local_dim} = e \times x \times y \times z \times t$ ；
3. 对每个粗网格块独立进行 QR 分解： $Q, R = \text{QR}(A^T)$ ，其中 $A \in \mathbb{C}^{\text{local_dim} \times E}$ ；
4. 可选地归一化 Q 的每一列： $Q = Q / \|Q\|_{\text{column}}$ ；
5. 恢复原始维度排列。

这一步骤的复杂张量重塑和置换在 Python 代码中通过 `reshape`, `permute`, `view` 组合实现，QR 分解使用 PyTorch 的 `torch.linalg.qr`（或 NPU 兼容的自定义路径）。

5.5 Galerkin 投影构建粗网格算子

粗网格算子的构造遵循 Galerkin 变分原理(`pyqcu/dslash/_operator.py::operator.__init__`，第 153-217 行)：

$$A_{\text{coarse}} = R A_{\text{fine}} P \quad (11)$$

其中 $P = \Phi$ （局域正交化后的近零空间向量矩阵）和 $R = \Phi^\dagger$ 分别是延拓（prolongation）和限制（restriction）算子。

代码实现中，粗网格算子的构造通过显式作用于每个粗基向量来完成。对每个粗方向的基向量 e_c （即粗空间中的一个单位向量）：

```

1  for e in range(coarse_dof): # 对每个粗自由度
2      for ward in range(4): # 对每个方向
3          # 构建指向特定粗格点和粗自由度的测试向量
4          _src_c = zeros_like(...)
5          _src_c[e][slice_dim(ward=ward, start=0)] = 1.0
6          # 延拓到细网格
7          _src_f = tools.prolong(local_ortho_null_vecs, _src_c)
8          # 在细网格上应用 hopping 算子
9          _dest_f_plus = fine_hopping.matvec_plus(ward, _src_f)
10         _dest_f_minus = fine_hopping.matvec_minus(ward, _src_f)
11         # 限制回粗网格
12         _dest_c_plus = tools.restrict(local_ortho_null_vecs, _dest_f_plus)
13         _dest_c_minus = tools.restrict(local_ortho_null_vecs, _dest_f_minus)
14         # 填充粗 hopping 和 sitting 矩阵

```

```

15     ...
16     # 对 sitting (Clover) 部分做同样处理
17     _src_c = ones_like(...)
18     _src_f = tools.prolong(local_ortho_null_vecs, _src_c)
19     _dest_f = fine_sitting.matvec(src=_src_f)
20     _dest_c = tools.restrict(local_ortho_null_vecs, _dest_f)
21     self.sitting.M[:, e] += _dest_c

```

这个过程将细网格算子的 hopping (on-site 部分和 hopping 部分) 和 sitting (*Clover*) 成分分别在每个方向上显式作用，然后通过限制操作投影到粗空间，构建出粗网格上的完整算子的矩阵表示。

5.6 延拓与限制操作

延拓 (prolongation) 和限制 (restriction) 操作在 `pyqcu/tools/_multigrid.py` 中实现，使用 Einstein 求和约定：

$$\text{Restrict: } v_{\text{coarse}}(E, X, Y, Z, T) = \sum_{e, x, y, z, t} \Phi^\dagger(E, e, Xx, Yy, Zz, Tt) \cdot v_{\text{fine}}(e, Xx, Yy, Zz, Tt) \quad (12a)$$

$$\text{Prolong: } v_{\text{fine}}(e, Xx, Yy, Zz, Tt) = \sum_{E, X, Y, Z, T} \Phi(E, e, Xx, Yy, Zz, Tt) \cdot v_{\text{coarse}}(E, X, Y, Z, T) \quad (12b)$$

CUDA 版本使用 `torch.einsum` 实现：

```

1 # Restrict
2 return _torch.einsum("EeXxYyZzTt, eXxYyZzTt->EXYZT",
3                       local_ortho_null_vecs.conj(), _fine_vec)
4 # Prolong
5 return _torch.einsum("EeXxYyZzTt, EXYZT->eXxYyZzTt",
6                       local_ortho_null_vecs, _coarse_vec)

```

NPU 版本 (不支持超过 8 维的 tensor) 采用分步 reshape+permute 策略，将高维操作拆解为多步低维操作。

6 V-cycle 递归求解与自适应策略

6.1 V-cycle 递归结构

MultiGrid 求解器的核心是 cycle 方法 (`pyqcu/solver/_multigrid.py:167-294`)，实现递归 V-cycle，如算法2所示。

V-cycle 的关键特征：

- **每层使用 BiCGStab 作为 smoother**：在每个层级上，BiCGStab 迭代充当平滑器 (smoother)，消除高频误差分量；

- **周期性粗网格校正**: 每 `num_restart` 步 (默认 5 步) 进行一次粗网格校正, 将当前残差限制到下一个粗层级并递归求解校正方程;
- **非对称 V-cycle**: 粗层级使用更大的容差 ($\|r\| \times 0.5$) 和最粗层级使用 $\|r\| \times 0.1$, 形成一种“指数松弛”策略。

6.2 自适应 Level-Back 机制

当收敛停滞时, `adaptive` 方法自动将有效层级数减少到 1 (即仅在最细层级上使用 BiCGStab), 避免粗网格校正无法有效改善收敛的情况。检测逻辑 (`pyqcu/solver/_multigrid.py:152-165`):

```

1 def adaptive(self, iter: int = 0):
2     if self.convergence_tol > 3:
3         self.num_level = 1 # 回退到仅最细层级
4     if (iter+1)*2 < self.num_convergence_sample+1:
5         return
6     convergence_now = self.convergence_history[-1]
7     convergence_sample = self.convergence_history[-(self.
8         num_convergence_sample+1):-1]
9     count = 0
10    for convergence in convergence_sample:
11        if convergence < convergence_now:
12            count += 1
13    if count >= self.num_convergence_sample // 2:
14        self.convergence_tol += 1 # 停滞计数递增

```

该机制监控最近 `num_convergence_sample` 次 (默认 50 次) 残差历史: 如果半数以上历史残差小于当前残差 (即收敛在变差), `convergence_tol` 递增; 当累积到 4 次以上停滞判断时, 强制回退到单层级 BiCGStab 求解。

7 CUDA C++ MultiGrid 参数体系

PyQCU 的 C++ define 头文件 (`cpp/cuda/qcu/include/define.h`) 中预留了完整的 MultiGrid 参数体系。参数通过 `params` 数组 (`_PARAMS_SIZE_ = 54`) 和 `argv` 数组 (`_ARGV_SIZE_ = 7`) 传递。

7.1 MultiGrid 相关参数索引

注: 当前版本 CUDA C++ 端的 MultiGrid 参数体系已定义但完整的多层级 CUDA 实现尚在开发中。目前 MultiGrid 的 V-cycle 逻辑在 Python 层实现, 仅在最细层级调用 CUDA 加速的 Clover BiCGStab Dslash。

表 2: MultiGrid 层级参数索引 (define.h)

宏定义	索引	含义
<code>_MG_NUM_LEVEL_</code>	17	MultiGrid 总层级数
<code>_MG_LEVEL_INDEX_</code>	18	当前层级索引
<i>Level 1 (最细层)</i>		
<code>_MG_LEVEL1_E_</code>	19	自由度数目 (近零空间向量数)
<code>_MG_LEVEL1_X/Y/Z/T_</code>	20-23	粗格点维度
<code>_MG_LEVEL1_MAX_ITER_</code>	24	最大迭代次数
<code>_MG_LEVEL1_DATA_TYPE_</code>	25	数据类型
<code>_MG_LEVEL1_NUM_RESTART_</code>	26	重启间隔
<i>Level 2-4 (类似)</i>		
<code>_MG_LEVEL2/3/4_E_ ...</code>	27-50	同上

表 3: MultiGrid argv 参数索引

宏定义	索引	含义
<code>_MASS_</code>	0	质量参数
<code>_ATOL_</code>	1	全局容差
<code>_SIGMA_</code>	2	σ 参数
<code>_MG_LEVEL1_ATOL_</code>	3	Level 1 容差
<code>_MG_LEVEL2_ATOL_</code>	4	Level 2 容差
<code>_MG_LEVEL3_ATOL_</code>	5	Level 3 容差
<code>_MG_LEVEL4_ATOL_</code>	6	Level 4 容差

8 Python-CUDA 桥接层 (Cython)

8.1 Cython 接口设计

PyQCU 使用 Cython (`pyqcu/cuda/qcu/qcu.pyx`) 作为 Python 与 CUDA C++ 之间的桥接层。所有 CUDA 函数接收裸指针 (封装为 `long long` 类型), 通过 Cython 直接传递 tensor 的底层数据指针:

```

1 # Python端调用
2 fermion_out_ptr = fermion_out.contiguous().data_ptr()
3 gauge_ptr = gauge.contiguous().data_ptr()
4 qcu.applyCloverBistabCgQcu(
5     fermion_out_ptr, fermion_in_ptr, gauge_ptr,
6     clover_ee_ptr, clover_oo_ptr,
7     clover_ee_inv_ptr, clover_oo_inv_ptr,
8     set_ptrs_ptr, params_ptr
9 )

```

C++ 端通过 `reinterpret_cast` 将 `long long` 恢复为指针。

8.2 参数传递

参数分为三类：

1. **params (int32, 54 维)**: 格点维度、网格维度、数据类型、迭代次数、plan 选择等整数参数；
2. **argv (float/complex, 7 维)**: 物理参数（质量、容差、 σ 等）；
3. **set_ptrs (int64)**: 预分配的指针槽位，用于存储 C++ 端创建的 `LatticeSet` 指针等运行时对象。

`pyqcu/cuda/define.py` 与 `cpp/cuda/qcu/include/define.h` 保持同步，定义了这些参数数组的索引常量。

8.3 Plan 系统

C++ 后端使用 Plan 系统区分不同的计算模式：

- `_SET_PLAN_N_2_` (-2): Laplacian 算子
- `_SET_PLAN_N_1_` (-1): Gauss 规范场生成
- `_SET_PLAN0_` (0): Wilson Dslash
- `_SET_PLAN1_` (1): BiCGStab/CG 及其 Dslash
- `_SET_PLAN2_` (2): Clover Dslash

不同 plan 分配不同大小的 scratch buffer(`LatticeSet::init` 中的条件分支),通过 `_SET_INDEX_` 参数区分同一 plan 下不同计算实例，避免 buffer 重用冲突。

8.4 生命周期管理

每次调用使用以下模式：

```
1 # 1. 初始化（分配 GPU 内存、创建 stream、copy 参数）
2 qcu.applyInitQcu(set_ptrs, params, argv)
3 # 2. 多次调用核心计算
4 qcu.applyCloverBistabCgQcu(...) # 或 applyCloverBistabCgDslashQcu
5 qcu.applyCloverBistabCgQcu(...) # 可以多次调用
6 # 3. 清理（释放内存、销毁 stream）
7 qcu.applyEndQcu(set_ptrs, params)
```

在 `applyInitQcu` (`apply_init.cu`) 中，C++ 创建 `LatticeSet<T>` 对象并调用 `init()`，分配所有必要的 GPU 内存和 CUDA 资源。该对象的指针存储在 `set_ptrs[_SET_INDEX_]` 中。后续调用通过该指针访问已初始化的资源。

8.5 MultiGrid 中的 CUDA 调用

在 MultiGrid 的 `init` 方法中 (`pyqcu/solver/_multigrid.py:65-145`)，CUDA 加速的 BiCGStab 被封装为一个闭包：

```
1 if self.with_cuda_qcu and i == 1: # 仅在 level 1 使用 CUDA
2     def _bistabcg(b, tol, max_iter, x0, ...):
3         fermion_in_eo = tools.ooxyz2pooxyz(b)
4         qcu.applyCloverBistabCgQcu(
5             fermion_out_eo, fermion_in_eo, gauge_eo,
6             clover_ee, clover_oo, clover_ee_inv, clover_oo_inv,
7             set_ptrs, params)
8         return tools.pooxyz2ooxyz(fermion_out_eo)
9     bistabcg = _bistabcg
```

这使近零空间向量的生成可以利用 CUDA 加速的求解器，随后粗层级算子构造在 Python 中完成。

9 性能优化策略总结

9.1 CUDA Kernel 级优化

1. **SU(3) 矩阵压缩存储**：仅存储前两行 6 个复数（12 个 float），第三行通过解析公式从反余子式重建，减少 33% 的全局内存带宽；
2. **寄存器驻留计算**：所有 spin-color 分量、邻点 src、SU(3) 矩阵均保持在寄存器中，dest 在寄存器中累加，避免共享内存/全局内存的中间写入；
3. **Kernel 融合**：四个方向的 hopping 操作在一个 kernel 中完成，避免多次 kernel 启动开销；
4. **Coalesced 内存访问**：一维线程索引映射到格点的一维展开，确保相邻线程访问相邻全局内存地址；
5. **Stream 并行**：Dslash 使用 4 个方向 stream 并行处理边界打包，BiCGStab 使用 4 个计算 stream 流水线化内积与向量更新。

9.2 MPI 通信优化

1. **计算-通信 Overlap**：内部格点 kernel 与边界数据 D2H 拷贝并行执行；
2. **Pinned Memory**：所有 MPI 缓冲区使用 `cudaMallocHost` 分配的页锁定内存，支持异步 DMA 传输；
3. **非阻塞 MPI**：支持 `MPI_Isend/MPI_Irecv` 非阻塞通信模式，可进一步增加 overlap；
4. **Clover 通信的 2D 面交换**：Clover 项构造需要跨角邻居数据，通过 6 个二维面(xy,xz,xt,yz,yt,zt) 的 4 通道 (BB,FB,BF,FF) `MPI_Sendrecv` 实现。

9.3 算法级优化

1. **Even-Odd 预条件**: 将条件数降低为原系统的平方根, 显著加速 BiCGStab 收敛;
2. **多层级粗化**: 粗化因子通常取 $2^4 = 16$, 每层的自由度从 12 递增到 24 或更大, 平衡粗网格算子的精度和求解成本;
3. **自适应 Level-Back**: 收敛停滞时自动回退, 避免粗网格校正无效时的浪费;
4. **混合精度策略**: 最细层级使用 complex64 (单精度), 粗层级使用 complex128 (双精度), 在精度和性能间取得平衡;
5. **设备异构计算**: 最细层级在 GPU 上执行 (利用 CUDA 加速), 粗层级可在 CPU 上执行 (粗层级数据量小, 通信开销可忽略)。

9.4 内存管理优化

1. **Clover 逆矩阵预计算**: 在 Python 层预先计算 A_{ee}^{-1} 和 A_{oo}^{-1} (12×12 矩阵的批量求逆), 避免每次迭代重复计算;
2. **Scratch Buffer 复用**: 通过 Plan 系统和 `_SET_INDEX_` 参数管理多组 scratch buffer 的生命周期;
3. **Lazy Memory Allocation**: C++ 后端使用 `cudaMallocAsync` 在指定 stream 上异步分配, 避免全局同步。

10 代码结构导航

表4给出了 MultiGrid 相关代码在 PyQCU 代码库中的完整导航。

11 总结与展望

本文对 PyQCU 中 CUDA C++ 版 MultiGrid 求解器的设计与实现进行了全面解析。系统采用“Python 控制 + CUDA 加速”的混合架构, 在 Python 层完成层级构造、近零空间生成、Galerkin 投影和 V-cycle 递归控制, 在 CUDA C++ 层提供高性能的 Wilson/Clover Dslash 算子和 BiCGStab 求解器。

当前实现的优化亮点包括:

- SU(3) 矩阵压缩存储与寄存器内计算;
- Even-Odd 预条件 Schur 补系统;
- 多 CUDA Stream 并行的 BiCGStab 实现;
- MPI 通信与计算的 Overlap 流水线;
- 自适应 Level-Back 收敛控制;

表 4: MultiGrid 相关代码位置

模块	文件路径
MG 求解器 (Python)	pyqcu/solver/_multigrid.py
MG 工具函数	pyqcu/tools/_multigrid.py
Dslash 算子	pyqcu/dslash/_operator.py
Wilson Dslash	pyqcu/dslash/_wilson.py
Clover 项	pyqcu/dslash/_clover.py
BiCGStab (Python)	pyqcu/solver/_bistabcg.py
Cython 桥接	pyqcu/cuda/qcu/qcu.pyx, qcu.pxd
参数定义	pyqcu/cuda/define.py
Wi Dslash Kernel	cpp/cuda/qcu/src/wilson_dslash.cu
Wilson Dslash MPI	cpp/cuda/qcu/include/lattice_wilson_dslash.h
Clover Dslash MPI	cpp/cuda/qcu/include/lattice_clover_dslash.h
Clover BiCGStab	cpp/cuda/qcu/include/lattice_clover_bistabcg.h
Wilson BiCGStab	cpp/cuda/qcu/include/lattice_wilson_bistabcg.h
Lattice Set 管理	cpp/cuda/qcu/include/lattice_set.h
核心 Define	cpp/cuda/qcu/include/define.h
初始化/清理	cpp/cuda/qcu/src/apply_init.cu, apply_end.cu
Clover BiCGStab 入口	cpp/cuda/qcu/src/apply_clover_bistabcg.cu
Clover Dslash 入口	cpp/cuda/qcu/src/apply_clover_bistabcg_dslash.cu
CMake 配置 (NVIDIA)	cpp/cuda/qcu/CMakeLists-nv.txt

- 混合精度与异构设备计算。

未来可能的优化方向包括：

1. **全层级 CUDA 加速**：将粗层级的 Dslash 和 BiCGStab 也移植到 CUDA，避免 CPU-GPU 数据传输；
2. **聚合型 AMG**：从当前的块 QR 正交化转向聚合型（aggregation-based）粗化，减少粗网格算子的构造开销；
3. **K-cycle/W-cycle**：实现更复杂的多重网格循环策略，提高粗网格校正效果；
4. **Tensor Core 加速**：利用 GPU Tensor Core 进行 SU(3) 矩阵乘法和 spinor 投影的混合精度计算；
5. **通信优化**：使用 CUDA-aware MPI（GPUDirect RDMA）消除 CPU staging buffer；
6. **自适应平滑器**：根据局部误差特性动态选择 Chebyshev、GMRES 或 MR 作为平滑器。

参考文献

- [1] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.
- [2] A. Frommer, K. Kahl, S. Krieg, B. Leder, and M. Rottmann, “Adaptive aggregation-based domain decomposition multigrid for the lattice Wilson–Dirac operator,” *SIAM J. Sci. Comput.*, vol. 36, no. 4, pp. A1581–A1608, 2014.
- [3] R. Babich, J. Brannick, R. C. Brower, M. A. Clark, T. A. Manteuffel, S. F. McCormick, J. C. Osborn, and C. Rebbi, “Adaptive multigrid algorithm for the lattice Wilson-Dirac operator,” *Phys. Rev. Lett.*, vol. 105, p. 201602, 2010.
- [4] M. A. Clark, R. Babich, K. Barros, R. C. Brower, and C. Rebbi, “Solving lattice QCD systems of equations using mixed precision solvers on GPUs,” *Comput. Phys. Commun.*, vol. 181, pp. 1517–1528, 2010.
- [5] B. Sheikholeslami and R. Wohlert, “Improved continuum limit lattice action for QCD with Wilson fermions,” *Nucl. Phys. B*, vol. 259, p. 572, 1985.
- [6] H. A. van der Vorst, “Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems,” *SIAM J. Sci. Stat. Comput.*, vol. 13, no. 2, pp. 631–644, 1992.

Algorithm 2 MultiGrid V-Cycle

```
1: function CYCLE(level)
2:    $b \leftarrow b_{\text{list}}[\text{level}]$ 
3:   if level == 0  $\wedge$  support_parity then
4:     切换到 Even-Odd 预条件系统:  $\hat{A}x_o = \hat{b}$ 
5:   end if
6:    $x \leftarrow 0, r \leftarrow b - Ax$ 
7:   if  $\|r\| < \text{tol}$  then return  $x$ 
8:   end if
9:   初始化 BiCGStab 变量:  $\tilde{r}, \rho, \alpha, \omega, p, v, s, t$ 
10:  for  $i = 0, \dots, \text{max\_iter} - 1$  do
11:    执行一步 BiCGStab 迭代 (更新  $x, r, \rho, \alpha, \omega$  等)
12:    count_restart  $\leftarrow$  count_restart + 1
13:    if level < num_level - 1  $\wedge$  count_restart > num_restart then
14:      // 粗网格校正
15:       $r \leftarrow b - Ax$  ▷ 重新计算残差
16:       $r_{\text{coarse}} \leftarrow \text{restrict}(r)$  ▷ 限制残差到粗网格
17:       $e_{\text{coarse}} \leftarrow \text{cycle}(\text{level} + 1)$  ▷ 递归求解粗网格校正
18:       $e_{\text{fine}} \leftarrow \text{prolong}(e_{\text{coarse}})$  ▷ 延拓校正到细网格
19:       $x \leftarrow x + e_{\text{fine}}$ 
20:       $r \leftarrow b - Ax$ 
21:      count_restart  $\leftarrow$  0
22:    end if
23:     $\|r\| \leftarrow \text{norm}(r)$ 
24:    if level == 0 then
25:      记录收敛历史; 执行自适应 level-back 检查
26:    end if
27:    if  $\|r\| < \text{tol}$  then return  $x$ 
28:    end if
29:  end for
30:  return  $x$ 
31: end function
```
